

ScheduleFree+: Scaling Learning-Rate-Free & Schedule-Free Learning to Large Language Models

Aaron Defazio¹

¹FAIR at Meta Super-Intelligence Labs

Schedule-Free Learning has shown promise as a practical anytime training method for machine learning, showing success across dozens of standard benchmark problems. However, strong performance for LLM training has only been demonstrated at small scales. We identify a number of fixes necessary to scale up Schedule-Free Learning to larger batch sizes and model sizes, and present a learning-rate-free and schedule-free method (ScheduleFree+) for training large language models which greatly outperforms Warmup-Stable-Decay (WSD) schedules. We also demonstrate that Schedule-Free Learning is most effective for long duration training, and at 1000 tokens per parameter, it outperforms SOTA schedules by 31%. Schedule-Free Learning provides a theoretical foundation for the use of model averaging and checkpoint merging during pretraining.

Date: May 20, 2026

Correspondence: adefazio@meta.com



1 Introduction

There should be no YOLO training runs. Machine Learning should be stable, smooth and predictable. ML Optimization research, recently laser-focused on building faster training methods, has recently seen a resurgence in methods whose goal is to obtain more reliable convergence with less tuning, all without compromising speed.

Schedule-Free Learning (Defazio et al., 2024) is a recently developed approach that aims to tackle these objectives. Traditional learning-rate schedules are completely replaced by an aggressive form of averaging. Averaging accomplishes the same convergence goal as scheduling, but produces far more stable and smooth loss curves, and gives highly predictable training behavior.

The switch from scheduling to averaging is an enormous change to the optimization process, requiring careful consideration of every other component of the optimizer. This tech report is a deep dive into the application of Schedule-Free Learning to Large Language Models. We identify a number of modifications necessary to match the performance of schedule-based approaches, and we show that Schedule-Free Learning particularly excels at long duration training runs, outperforming schedule-based baselines at 1000 tokens per parameter, giving a *31% reduction in training time* to reach the same loss. No final loss advantage is seen for short duration (20-100 TPP) runs, although the anytime stopping capability of Schedule-Free Learning is useful at any duration.

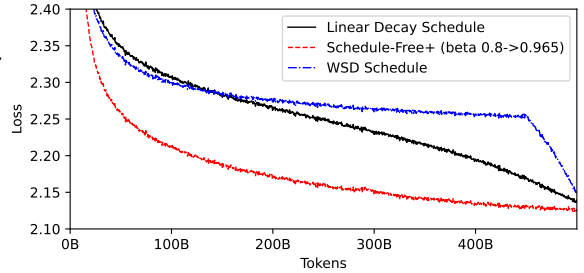


Figure 1 1000 tokens per parameter run with a 500M parameter model

We identify a number of modifications necessary to match the performance of schedule-based approaches, and we show that Schedule-Free Learning particularly excels at long duration training runs, outperforming schedule-based baselines at 1000 tokens per parameter, giving a *31% reduction in training time* to reach the same loss. No final loss advantage is seen for short duration (20-100 TPP) runs, although the anytime stopping capability of Schedule-Free Learning is useful at any duration.

Summary of Key Ideas

1. Reintroduction of inner momentum into Schedule-Free training fixes large-batch training divergence.
2. A proposed variant of the Polyak step size gives a practical learning-rate-free optimizer for language model training.
3. Warm-starting Schedule-Free further improves early loss values.

4. Using increasing outer-momentum in Schedule-Free gives large improvements for long training runs.

We call the combination of these approaches ScheduleFree+. We see large improvements in loss compared to WSD Schedules and even compared to state-of-the-art Linear Decay schedules.

2 Schedule-Free Learning

Consider a base optimizer that updates the sequences of iterates z_t using a step direction G computed from the stochastic gradient at z_t :

$$z_{t+1} = z_t - \gamma_t G_t(z_t). \quad (1)$$

Here G could be the step direction of AdamW, Muon or SGD for example. Schedule-Free Learning modifies the learning process by introducing an average iterate buffer x_t , that computes an online average of z_t :

$$x_{t+1} = \frac{t}{t+1}x_t + \frac{1}{t+1}z_{t+1}, \quad (2)$$

so that $x_T = \frac{1}{T} \sum_{t=1}^T z_t$. The idea of maintaining an average point is fundamental to modern optimization theory. Classical convex convergence analysis actually focuses on the average iterate x , NOT z , however x by itself converges very poorly on neural network training runs. The key enabling idea that takes averaging from unusable to state-of-the-art, is to compute new gradients at a point y that *mixes in* the average, using an interpolation between the averaged and non-averaged points:

$$y_t = (1 - \beta) z_t + \beta x_t. \quad (3)$$

Typically, values of $\beta = 0.9$ work well, so that the y sequence is mostly x , with a small amount of more recent information from z mixed in. The full method is as follows:

$$\begin{aligned} y_t &= (1 - \beta) z_t + \beta x_t, \\ z_{t+1} &= z_t - \gamma_t G_t(y_t), \\ x_{t+1} &= (1 - c_{t+1}) x_t + c_{t+1} z_{t+1}, \end{aligned}$$

This formulation, due to Defazio et al. (2024), includes a more general averaging coefficient c_{t+1} , which they set to match $1/t$ eventually, but which provides a more front-weighted average during the learning rate warmup period. The y sequence doesn't need to be stored between steps, it can be computed as needed on each step.

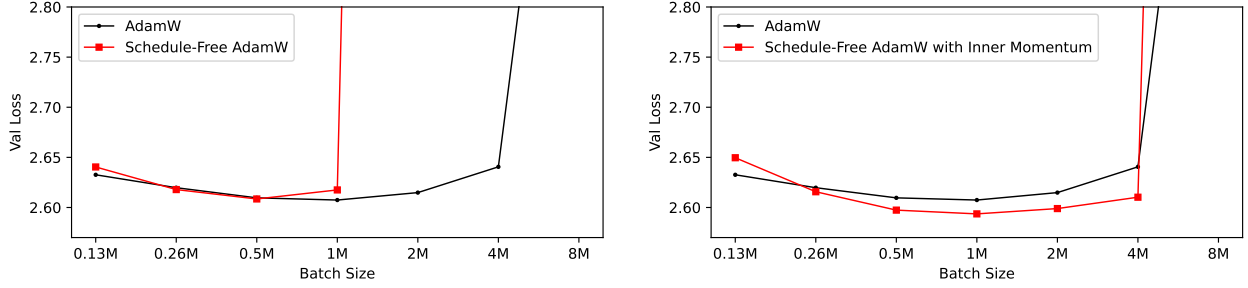
Schedule-Free Learning changes one of the standard aspects of optimizer APIs common to prior methods: the points where you evaluate the gradient, y , are no longer your best model estimate. Loss values for the x sequence are much lower, the latest x_t is the model that should be used for evaluations and ultimately returned by the method as output. This can require additional code outside of the usual `.step()` interface.

The use of averaging means that a decreasing step size is not necessary with Schedule-Free Learning, however a learning rate warmup is still needed for best performance.

3 Handling Larger Batch-Sizes

Recent evaluations of Schedule-Free Learning on LLM pretraining show that it significantly outperforms Cosine and WSD schedules at medium batch-sizes. However, as the batch-size is increased it falls behind (Zhang et al., 2024; Morwani et al., 2025). This behavior was not identified in Defazio et al. (2024) as training was performed on V100 GPUs which can only handle relatively small batch-sizes.

In Figure 2, we investigate the behavior of Schedule-Free AdamW against AdamW with a Linear-Decay Schedule (Defazio et al., 2023) as the batch-size is varied. This task uses a 120M parameter Llama 3 architecture transformer trained on the FineWeb-EDU dataset for 8B tokens. Warm-starting from the same 2B token checkpoint is used for all runs, which provides better performance for large-batch sizes. For both methods, the learning rate was tuned at batch-size 1M, and set using a $\sqrt{b}$ scaling law for other batch-sizes. This



(a) Standard Schedule-Free Learning is not robust to increasing batch-sizes (b) Inner loop momentum results in greater robustness to larger batch-sizes

Figure 2 Batch-size scaling on language model pretraining

square-root scaling law for learning rate given batch-size is well-established, and we confirmed experimentally that it outperforms linear scaling on this problem. This scaling is not perfect, and for the smallest batch-size we see worse loss values for Schedule-Free than can be obtained by manually tuning at that batch-size.

Figure 2a shows that Schedule-Free falls behind in terms of loss for larger batch-sizes, and hits a scaling “cliff” at 2M tokens per batch, compared to AdamW which has a similar cliff at 4M tokens.

This behavior can be fixed by **reintroducing inner momentum** into Schedule-Free Learning. Defazio et al. (2024) removed the momentum from AdamW when applying Schedule-Free Learning on top of it as it did not provide any benefit in their early experimentation on small models, and by removing inner momentum they were able to reduce the memory usage of Schedule-Free Learning.

Figure 2b shows that Schedule-Free AdamW with inner momentum (here $\beta_1 = 0.75$) performs better at small batch-sizes, while also allowing the use of batch-sizes of the same magnitude as regular AdamW.

Why does inner momentum help with larger batch-sizes?

Momentum has been shown experimentally to help with large-batch training, while providing little or no improvement at small batch-sizes. Shallue et al. (2019) show this effect clearly for vision models, and Marek et al. (2025) is a more recent comparison using a LLM pretraining task, showing that this phenomenon is surprisingly universal. Zhang et al. (2019) established that exactly this behavior can be proven to hold under a noisy-quadratic model of the objective function.

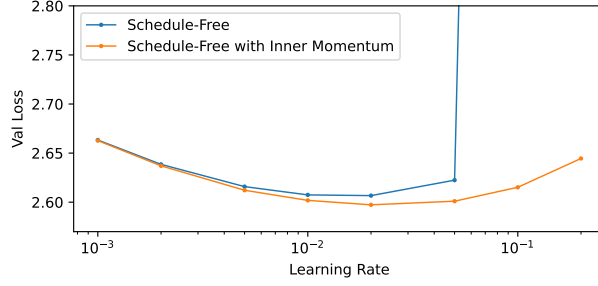
We believe this behavior is due to enhanced robustness to larger step-sizes, rather than to larger batch-sizes, as momentum allows stable training under larger step-sizes even for smaller-batch sizes. As the batch-size is increased, the optimal learning rate increases, and that optimal learning rate becomes unstable earlier when no momentum is used. This is shown in Figure 3a for training at batch-size 0.5M with traditional Schedule-Free ($\beta_1 = 0$) and with Inner momentum ($\beta_1 = 0.9$). Training is stable and achieves the same loss for smaller learning rates, however the zero-momentum version starts to have worse loss and then diverges as higher learning rates are used.

Investigating C-Refinement for large batch training

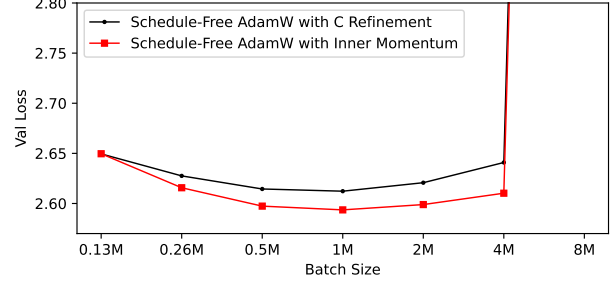
Song et al. (2025) propose a fix for large-batch training that modifies the averaging weights. They propose modifying the averaging weights for x in the update $x_{t+1} = (1 - c_{t+1})x_t + c_{t+1}z_{t+1}$ to:

$$c_{t+1} = \frac{(1 - \beta)C}{t + 1}. \quad (4)$$

Where C is a new hyper-parameter constant. When rewriting the update of x purely in terms of y (i.e. eliminating z), this version removes the dependence of the averaging weights for x on β . Defazio et al. (2024)

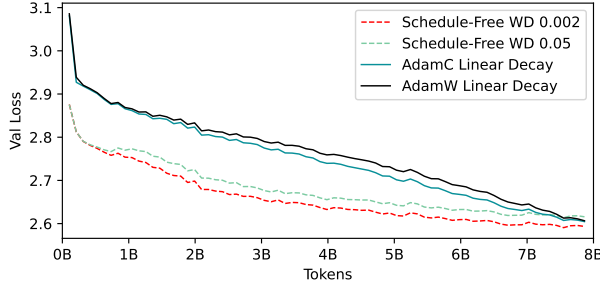


(a) Inner momentum of $\beta_1 = 0.9$ enables training with larger learning rates

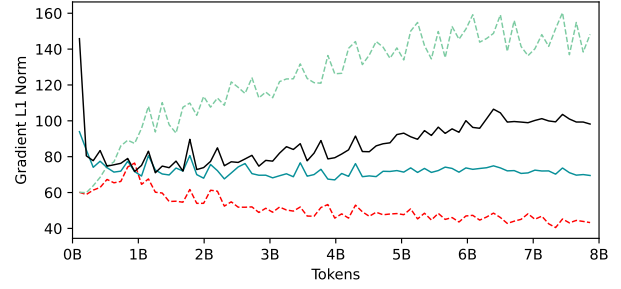


(b) Using inner momentum gives lower loss at larger batch sizes than using C-Refinement

Figure 3 Inner momentum comparison



(a) Larger "normal" levels of weight decay are harmful for Schedule-Free Learning



(b) Unexpectedly, gradient norms are flatter for smaller WD values for Schedule-Free Learning

Figure 4 The effect of weight-decay on gradient norms

discuss a tunable hyper-parameter r which can also be used to control the weighting sequence, via the update:

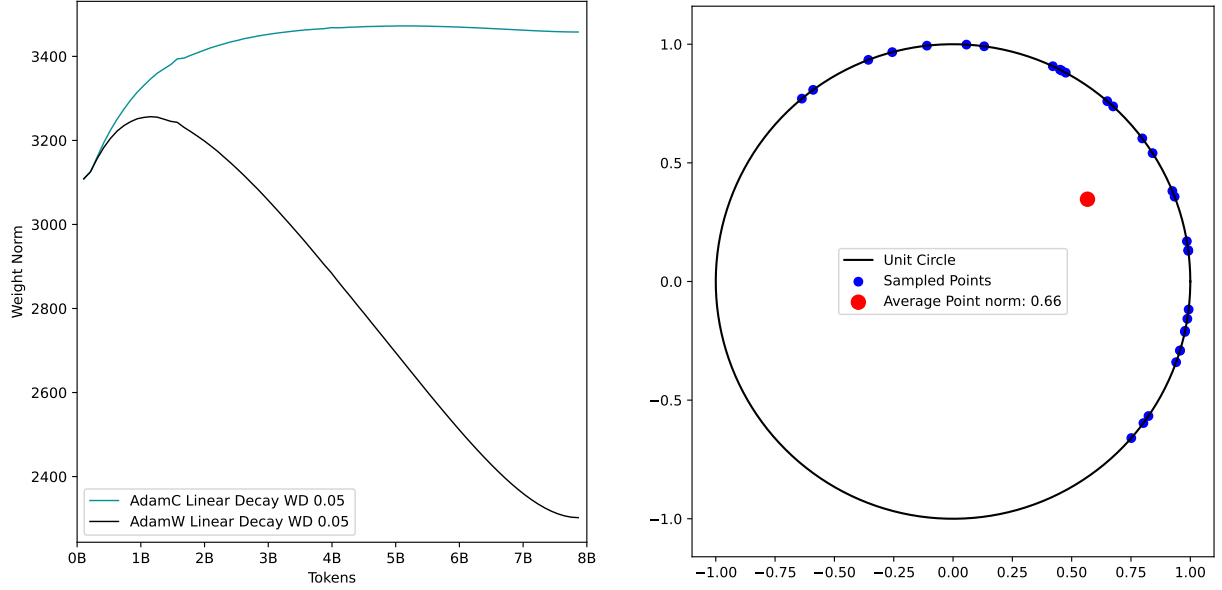
$$c_{t+1} = \frac{r + 1}{t + r + 1}. \quad (5)$$

The C hyper-parameter can be seen as performing a similar function to r but parameterized differently. The reference implementation uses a different, approximate implementation of r -weighting, which we discuss further in Section 8. We implemented the C version as given by Song et al. (2025) and tested it for large batch-training at $C = 50$ and $C = 200$. For this test problem, $C = 50$ performed slightly better. Figure 3b shows that using C-Refinement does enable training at larger batch sizes, but is not as effective as our inner momentum approach.

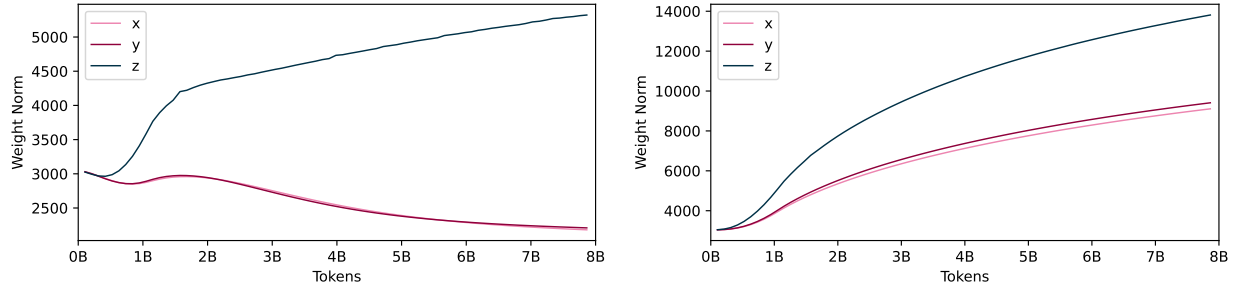
4 Handling Gradient Norm Drift

In Defazio et al. (2024), best results are only obtained with unusual values of the weight-decay hyper-parameter. The weight-decay used is much smaller than typical for AdamW. While these settings work, they induce undesirably large weight norms in trained models, particularly for long training runs. In this section, we show that these smaller weight-decay values are actually compensating for gradient norm drift, in a somewhat hacky way for which better solutions exist.

The key clue to understanding this odd behavior lies in the change that occurs in the gradient norms during training. Figure 4b shows that Schedule-Free Learning with weight-decay 0.002 produces a more stable gradient norm sequence over time than weight-decay 0.05, which more than doubles during the course of training. In comparison, regular AdamW with a Linear Decay schedule only shows a 25% change in gradient norm (outside of the initial warmup period), and AdamC, a small modification to AdamW designed to produce flatter gradient norm sequences (Defazio, 2025), has nearly constant gradient norm.



(a) Linear Decay Schedules with WD 0.05: AdamC produces more stable norms (b) Averages have lower weight norms when weights are on the unit sphere



(c) Schedule-Free with WD 0.05: Weight norms decrease for the x/y sequences (d) Schedule-Free with WD 0.002 has faster increasing weight norms

Figure 5 Schedule-Free Learning induces different weight norm dynamics than traditional schedules

Constant or near-constant gradient norms during training appears to be empirically beneficial during training. Defazio (2025) show that AdamC provides clear benefits, particularly for longer training runs. Figure 4a shows that AdamC slightly outperforms AdamW here, but the difference is far larger for Schedule-Free Learning, where a large gap in loss is observed between the two Schedule-Free Learning runs.

The underlying reason for this behavior is a simple consequence of the use of normalization layers. Van Laarhoven (2017) show that networks trained with normalization layers have a different “effective learning rate” for each layer that depends inversely on that layer’s weight norm. When training with SGD, the effective learning rate is:

$$\eta_{\text{eff}} \propto \eta_t / \|w_t\|^2. \quad (6)$$

Here the $\propto$ notation indicates proportional scaling. Gradient norms depend inversely on weight norms (Kosson et al., 2024; Defazio, 2025), i.e. $\|w_t\| \propto \|g_t\|^{-1}$, and so the effective learning rate is proportional to the gradient norm squared:

$$\eta_{\text{eff}} \propto \eta_t \|g_t\|^2, \quad (7)$$

and so an increasing gradient norm indicates an increasing effective learning rate. The optimization theory supporting both Schedules and Schedule-Free Learning suggests that a flat or decreasing learning rate (modulo the schedule) is optimal, and so this increasing effective learning rate can be expected to hurt performance. Although this argument is for SGD, similar arguments suggest that the effective learning rate varies as

$\eta_{\text{eff}} = \eta_t \|g_t\|_1$ for Adam-like methods. The gradient magnitude estimates used in the denominator of the Adam step are NOT enough to correct the learning rate.

Now, as we observed in Figure 4b, Schedule-Free Learning with small weight decay values appears empirically to give flatter gradient-norm sequences. This behavior appears to be induced by growing weight-norms counter-acting the gradient-norm increase. Figure 5c shows that larger WD values induce decreasing weight norms in the x, y sequences used in Schedule-Free Learning, whereas smaller WD values (Figure 5d) give increasing weight norms. We find that when tuning the method to give the best loss, the weight-decay values inducing approximately flat gradient norm sequences give the best loss values.

Rapidly increasing weight norms are undesirable as they increase instability and cause issues with weight quantization. We found unpredictable and time-horizon dependent behavior when trying to scale to large model sizes and training durations when using small weight-decay values. Instead, flat weight-norms, induced by using larger weight decay values, result in more predictable behavior, but require special handling when using Schedule-Free Learning.

When using flat or near-flat learning rate sequences, weight-decay induces weight norms to lie on a hyper-sphere (Defazio, 2025), overriding values from initialization (Kosson et al., 2025). However, when using iterate averages as used in Schedule-Free Learning, the average point will have a smaller norm. This is illustrated schematically in Figure 5b in 2D. The average of points on the surface of the sphere will be in the interior of the sphere. This same phenomenon is seen in practice, where the average point’s weight norm drops by roughly a factor of 2 as the time progresses before stabilizing (Figure 6c).

This is a key problem that must be addressed when applying Schedule-Free Learning: **Shrinking weight norms induces increasing gradient norms in the model, which affects the effective learning rate** (Equation 7). In the next section we explore the corrections required to fix the convergence of Schedule-Free Learning in this setting.

5 Inverse-Gradient Norm Weighting is Highly Beneficial

The theory behind Schedule-Free Learning provides a (anytime) last-iterate convergence guarantee for the x_T sequence that depends on the weighted regret of the base optimizer. In practice, the weights used are typically just the learning rates γ_t , and the bound is as follows:

$$\mathbb{E}[F(x_T) - F(x_*)] \leq \frac{\mathbb{E}[\sum_{t=1}^T \gamma_t \langle g_t, z_t - x_* \rangle]}{\sum_{i=1}^T \gamma_i}. \quad (8)$$

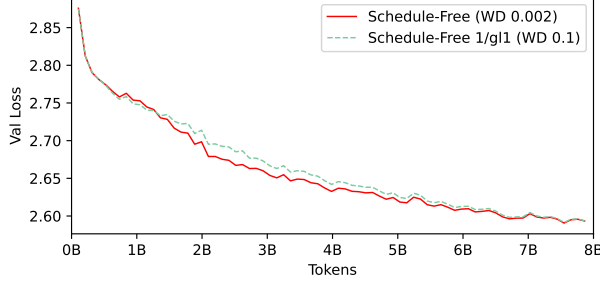
Here g_t is the (sub-)gradient at time-step t , and x_* is any minimizer of the (convex) objective function F . As an example, we can apply this to the SGD step $x_{t+1} = x_t - \gamma_t g_t$, which has γ -weighted regret of (Zinkevich, 2003):

$$\sum_{t=1}^T \gamma_t \langle g_t, z_t - x_* \rangle \leq \frac{1}{2} D^2 + \frac{1}{2} \sum_{t=1}^T \gamma_t^2 \|g_t\|^2.$$

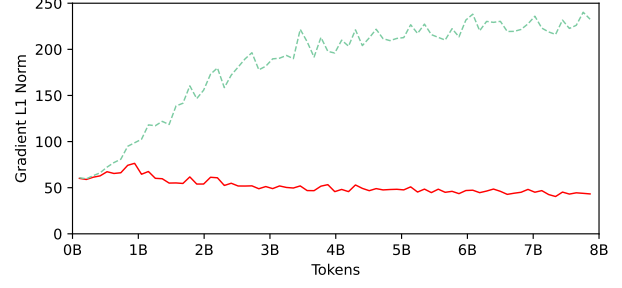
The benefit of a bound that depends on weighted-regret rather than standard unweighted-regret is that we have additional flexibility to optimize the right hand side of the bound by optimizing the weights in a problem dependent way, with the caveat that the Schedule-Free theory requires the weight at time t to be statistically independent of the sampling noise at time t . The optimal weights, if the gradient-norm sequence is fixed in advance, are given by the following theorem:

Theorem 1. (Defazio et al., 2023) *For a fixed sequence $\|g_1\|^2, \dots, \|g_T\|^2$, the value of $\frac{1}{2 \cdot \sum_{t=1}^T \gamma_t} (D^2 + \sum_{t=1}^T \gamma_t^2 \|g_t\|^2)$ is minimized by setting γ_t proportional to $\|g_t\|^{-2}$.*

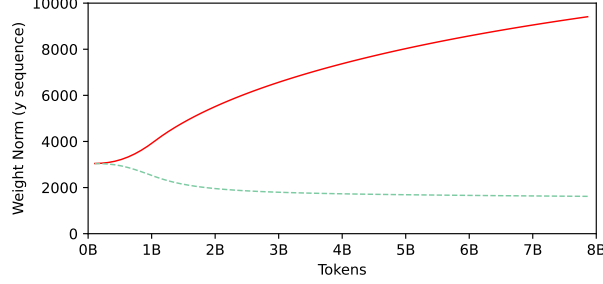
We may approximate this ideal weighting while dodging issues of statistical-independence by using an estimate of the gradient norm that is temporally smoothed, which will still capture the overall scale of the gradient-norm. An exponential-moving average (EMA) of the gradient norm, such as used for a momentum buffer (with dampening), provides such an estimate.



(a) Loss values with and without inverse-gradient norm weighting.



(b) Gradient Norms differ substantially, rising higher then stabilizing when larger WD values are used.



(c) Weight norms for Schedule-Free with larger WD values stabilize after an initial burn-in period.

Figure 6 Inverse gradient L1 norm step sizes give equivalent final loss to using small WD values, while having a stable weight-norm during training

Notice that this result suggests a correction to the learning rate that counter-acts the gradient-norm-squared drift in the learning rate produced by Normalized layers (Equation 7). Although the behavior of normalized layers is non-convex in nature, the above analysis in the convex case suggests the exact same correction!

In the case of Adam, the weighting formula changes to:

$$\gamma_t \propto \left(\sum_{i=1}^d \frac{g_{t,i}^2}{\sqrt{v_{t,i}}} \right)^{-1} \approx \left(\sqrt{\frac{\pi}{2}} \|g_t\|_1 \right)^{-1}. \quad (9)$$

Here d is the dimension of the vectors, $g_{t,i}$ the i th element of g_t , and v is the EMA of the square of the gradients maintained in Adam. The $\sqrt{\pi/2} \approx 1.27$ factor comes from converting from squares to absolute values under a normality assumption:

$$E[|x|] = \sqrt{\frac{\pi}{2}} \sigma \quad \text{when } x \sim \mathcal{N}(0, \sigma^2).$$

This approximation is very accurate in practice and is far more stable than directly using g and v to compute the scaling factor (Figure 7).

This correction is *highly* effective in practice, as demonstrated in Figure 6. Here we weight by a 0.9 parameter EMA using the inverse (non-squared) gradient L1 norms. The numerator of the learning rate was tuned by grid search. We see that the final validation loss matches our previous best result for the unweighted case, but now we entirely eliminate the issue of rising weight norms, allowing for the training of larger models, and for longer durations without issue.

So we recommend that **Schedule-Free step-sizes for Adam should be scaled inversely proportional to the L1 norm of the gradient:**

$$\gamma_t \propto \frac{1}{\|g_t\|_1}. \quad (10)$$

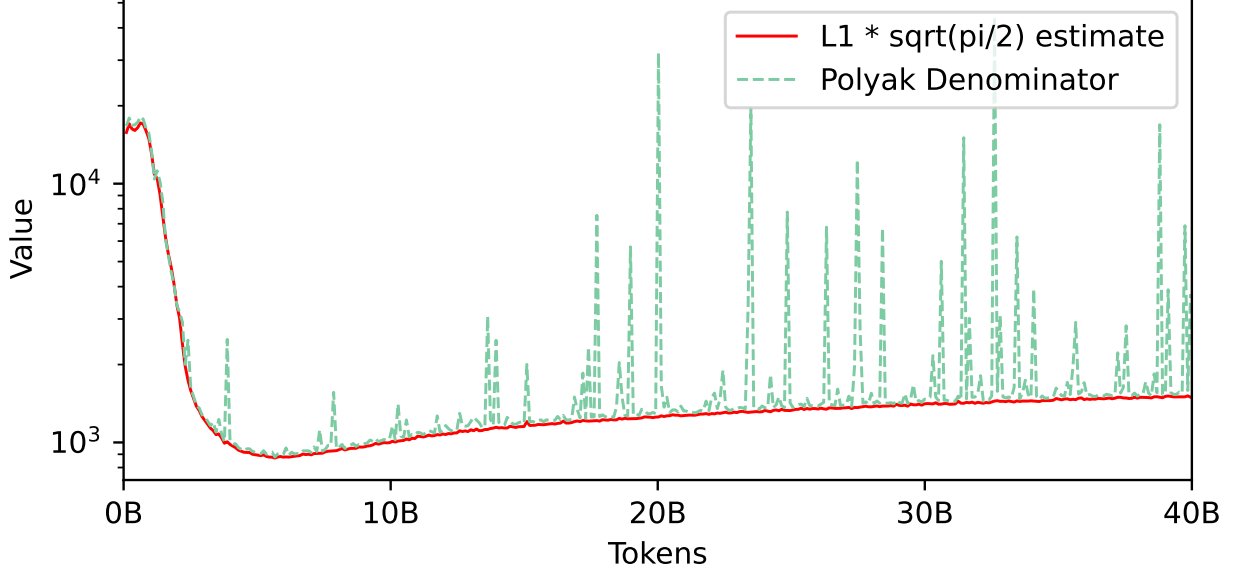


Figure 7 Using the L1 norm approximation to the Polyak denominator gives an accurate and low variance estimate. Training of a 2B model for 40B tokens is shown, with a 0.9 weight EMA used on both sequences. Direct estimation of the Polyak denominator is too noisy to be used in practice.

6 Learning-Rate Free Learning of LLMs

Inverse-gradient norm weighting is a key component of the popular Polyak Step size for deterministic optimization:

$$\gamma_t = \frac{f(x_t) - f_*}{\|\nabla f(x_t)\|^2}. \quad (11)$$

It’s curious that the same inverse-squared gradient weighting appears in the Polyak step as appears in optimal schedules, and the effective learning rate of normalized layers. This suggests that by using the Polyak step size, we can avoid the need to do any additional inverse-gradient norm weighting. It’s unclear if there is any deeper connection between these three appearances of the same weighting.

The Polyak step size can be applied to Schedule-Free AdamW training using the approximation in Equation 9 for the denominator:

$$\gamma_t = \frac{f(y_t) - f_* + \beta \langle \nabla f(y_t), z_t - x_t \rangle}{\sqrt{\frac{\pi}{2}} \|\nabla f(y_t)\|_1}. \quad (12)$$

The numerator also contains a correction factor, which approximates the unknown quantity $f(z_t) - f_*$ which should appear in the numerator, using the Taylor expansion of f around the point y_t as introduced by [Oikonomou et al. \(2026\)](#):

$$\begin{aligned} f(z_t) &\approx f(y_t) + \langle \nabla f(y_t), z_t - y_t \rangle \\ &\approx f(y_t) + \langle \nabla f(y_t), z_t - (\beta x_t + (1 - \beta)z_t) \rangle \\ &\approx f(y_t) + \beta \langle \nabla f(y_t), z_t - x_t \rangle. \end{aligned}$$

When applied to stochastic problems, the gradients and function values need to be estimated. We focus on the simplest approach to estimating these quantities: We apply an exponential moving average (coefficient 0.9) to the stochastic estimates of each quantity.

Polyak step sizes have been heavily investigated for their potential in machine learning in the last few years, as they have appealing theoretical properties, however a number of issues arise in their application to stochastic and non-convex optimization problems. In practice, these issues have led to the Polyak step size not being

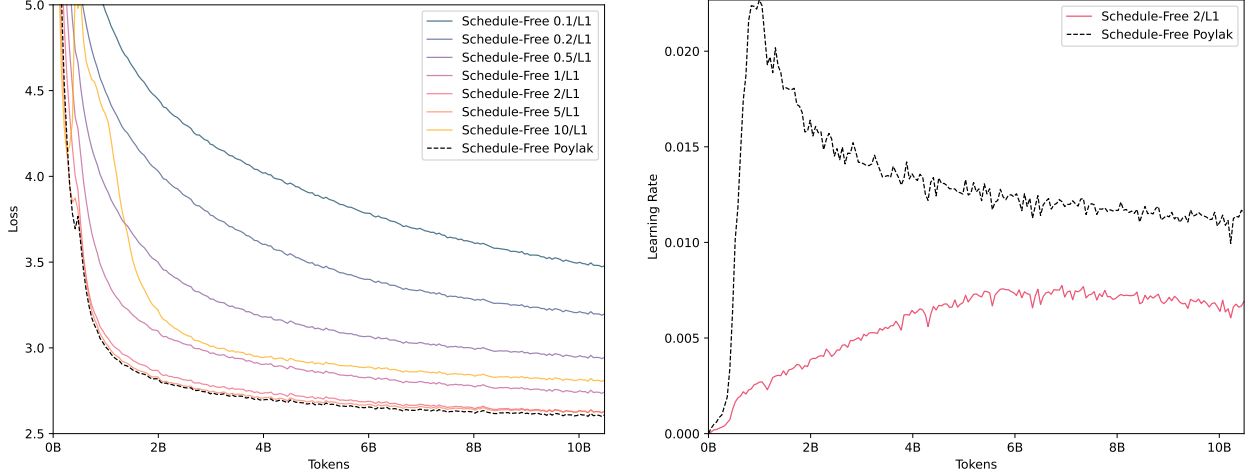


Figure 8 Polyak step sizes outperform a tuned grid search over L1-adjusted step-sizes

competitive against other adaptive learning rate methods such as Prodigy (Mishchenko and Defazio, 2023). The use of Schedule-Free Learning appears to ameliorate each of these issues, resulting in a practical step-size rule.

Vanishing denominators Small gradients in the denominator result in a blow-up of the step-size, destabilizing training. This can occur simply due to gradient noise during training. Using large-batch sizes greatly help with this issue, but small gradients still occur when small learning rates are used near the end of training when using a classical learning rate schedule. Schedule-Free avoids this issue by maintaining large learning rates throughout training.

Sensitivity to f_* When using schedule-based training, the Polyak numerator uses the current iterate x_t in the numerator (Equation 11). This quantity is noisy, and this noise gets worse as (x_t) approaches f_* during training, giving unstable learning rates. For Schedule-Free Learning, an estimate of $f(z_t)$ is used instead, which is much larger than $f(x_t)$ while having similar noise levels, and therefore lower absolute error. In practice we find that $f(z_t) - f_*$ is a stable quantity to estimate.

The Polyak step size also correctly scales the learning rate as batch-size is varied. When using SGD, the squared-gradient denominator results in a doubling of the learning rate as the batch-size is doubled, following established linear scaling rules. When using AdamW, the learning rate automatically scales proportionally to the square-root of the batch-size, again following standard scaling rules. The Polyak step size also scales the step-size down as training progresses, meaning that no tuning is needed as the number of training tokens is increased.

Figure 8 shows the results of using the Polyak step size with 0.4B tokens of warmup (A linear multiplicative warmup), compared against the $\gamma/L1$ adjusted step sizes with a grid-search of γ shown, using the same warmup duration. The Polyak step size outperforms the results of the grid search, and generally uses larger step-sizes than the best performing $\gamma = 2$ learning rate. This appears to be due to a higher initial learning rate that comes from the numerator of the step size. **The Polyak step-size automatically sets the learning rate as effectively as a tuned grid search** and provides the necessary gradient L1-norm adaptivity automatically.

6.1 Learning Rate Adaptation Breaks Weight Decay

In neural networks using normalization layers, weight decay’s (λ) primary effect is to control the eventual magnitude of gradient-to-weight norm ratio (Van Laarhoven, 2017):

$$\frac{\|g_t\|}{\|z_t\|} = \sqrt{\frac{2\lambda}{\gamma_t}}.$$

The gradient/weight ratio can be thought of as a simple dynamical system that rapidly approaches this steady state value. When the learning rate is changing adaptively during training, we end up with a coupled dynamical system, where changes to the learning rate, *feedback into a changing gradient norm*, which then affects the learning rate estimate, and so forth, resulting in erratic, cyclic and potentially divergent behavior.

This undesirable feedback loop WILL break learning rate adaptation, and we believe it is the primary cause of the failure of adaptive learning rate methods over the last decade. The exception is recently developed methods that only produce non-decreasing learning rate sequences, which can not cycle erratically, such as D-Adaptation (Defazio and Mishchenko, 2023), Prodigy (Mishchenko and Defazio, 2023), DoG (Ivgi et al., 2023) and DoWG (Khaled et al., 2023).

Defazio (2025) suggests a simple fix for this, modifying the AdamW step to multiply the weight-decay term by an additional *extra* $\gamma/\gamma_{\max}$:

$$z_{t+1} = z_t - \gamma_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \frac{\gamma_t^2}{\gamma_{\max}} \lambda z_t. \quad (13)$$

This squared learning rate is counter-intuitive, but a short calculation shows that it results in a steady state independent of γ_t :

$$\frac{\|g_t\|}{\|z_t\|} = \sqrt{\frac{2\lambda}{\gamma_{\max}}}.$$

When using adaptive learning rates, there is no notion of $\gamma_{\max}$ so we leave out that term, and use *fully-decoupled AdamC*:

$$z_{t+1} = z_t - \gamma_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \gamma_t^2 \lambda z_t. \quad (14)$$

Giving a steady state of:

$$\frac{\|g_t\|}{\|z_t\|} = \sqrt{2\lambda}.$$

The purpose of the $\gamma_{\max}$ term was to keep the λ hyper-parameter on the same scale as regular AdamW. Consequently, Schedule-Free with this modification will require different, larger, weight-decay values, typically in the range 5-50. **Using fully-decoupled AdamC is necessary when using the Polyak step size.**

7 Warming-up Schedule-Free

The original Schedule-Free formulation applied iterate averaging from the beginning of optimization. The weight of each iterate in the average was chosen as proportional to the learning rate squared, which resulted in points during the learning rate warmup phase having lower weight. This approach is very general, however in the case of models where the norm of weights changes dramatically during the early stages of training, this procedure is suboptimal, resulting in slower initial convergence.

There are several ways to remedy this. Weights can be initialized closer to their final steady state norms. Weight projection can be used, so that weight norms do not change during training. Training can be warm-started from a model trained until weight norms are stable, using a constant learning rate. No fix is needed at all for long training runs. We opted to implement a simple solution, where iterate averaging is not performed for an initial fixed number of steps. This is implemented by setting $c_t = 1$ in the Schedule-Free update equations.

Figure 9 shows the results of using c_t equal to 1 for t up to 200, 400 and 800 steps, when a learning rate warmup of 400 steps is used. c warmup of 800 steps gave the lowest loss in this example. **A good heuristic is to do c warmup for 2x the length of the learning rate warmup.** Longer c warmup does introduce a “hump” after iterate averaging is introduced, but final loss is significantly lower.

8 Benefits of Annealing and Weighting

The β parameter in Schedule-Free Learning interpolates between the raw iterate sequence z_t and its running average x_t . This parameter controls a trade-off between the recency of information (the z_t sequence is updated

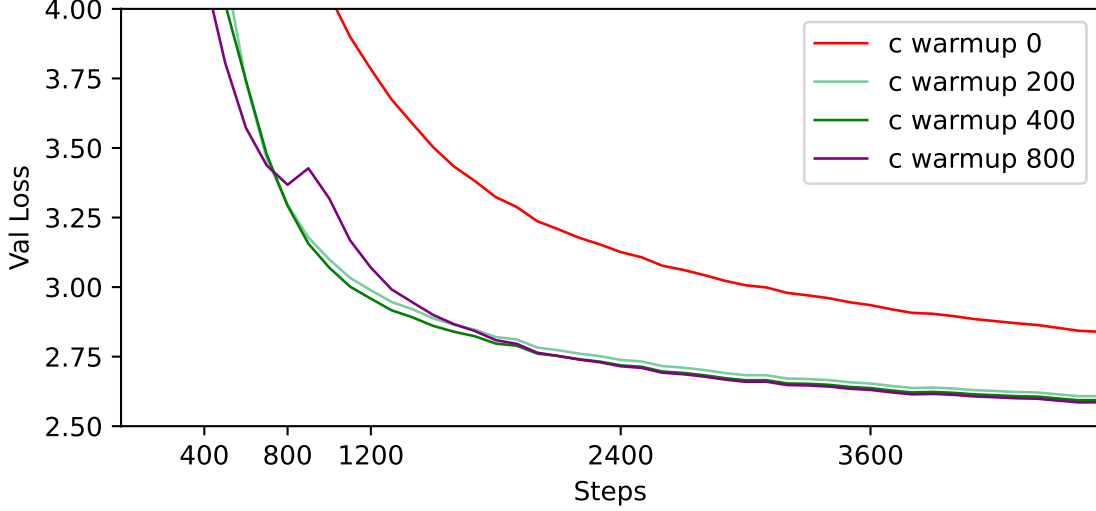
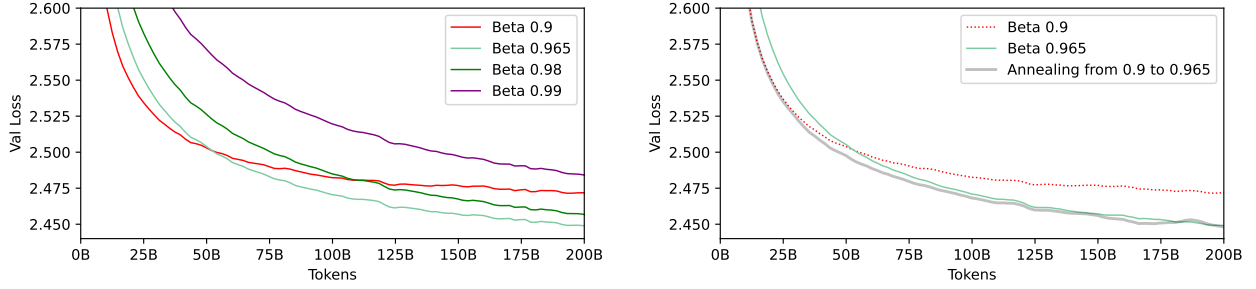


Figure 9 Warm-starting Schedule-Free Learning significantly improves loss values



(a) Smaller β values are beneficial at the early stages of training, whereas larger values around 0.965 work better at later stages.

(b) Slowly Annealing β during the course of training gives a best-of-both-worlds behavior. Here β is log-linearly interpolated from 0.9 to 0.965 over the course of the run.

Figure 10 Schedule-Free β experiments

immediately with new gradient information) and the level of smoothing (as an average, the x_t sequence changes slowly).

Figure 10a shows a sweep of the β parameter to illustrate this behavior. Low beta values converge faster initially but have worse long run performance. This is another behavior that is not witnessed on smaller scale deep learning problems (Defazio et al., 2024) but clearly visible for long duration LLM training runs.

Fortunately, we can obtain both rapid early convergence as well as ideal later stage convergence by *annealing* the beta parameter over time. In Figure 10b we use a time-varying β given by a log-linear interpolation:

$$\beta_t = 1 - \exp \left(\left(1 - \frac{t}{T} \right) \log(1 - \beta_{\text{initial}}) + \frac{t}{T} \log(1 - \beta_{\text{final}}) \right). \quad (15)$$

This results in a smooth interpolation between the two β values behavior, capturing the fast initial drop in loss with β_{initial} and the lower later-stage loss matching $\beta_{\text{final}} = 0.965$. For long duration training runs, **annealing Schedule-Free β from smaller values like 0.9 to 0.965 gives great loss values throughout the training run.**

The reference implementation of Schedule-Free Learning includes a parameter r that controls the weights of points within the weighted average x . The weight at time t is given by:

$$w_t = t^r \max_{i \leq t} \gamma_i^2. \quad (16)$$

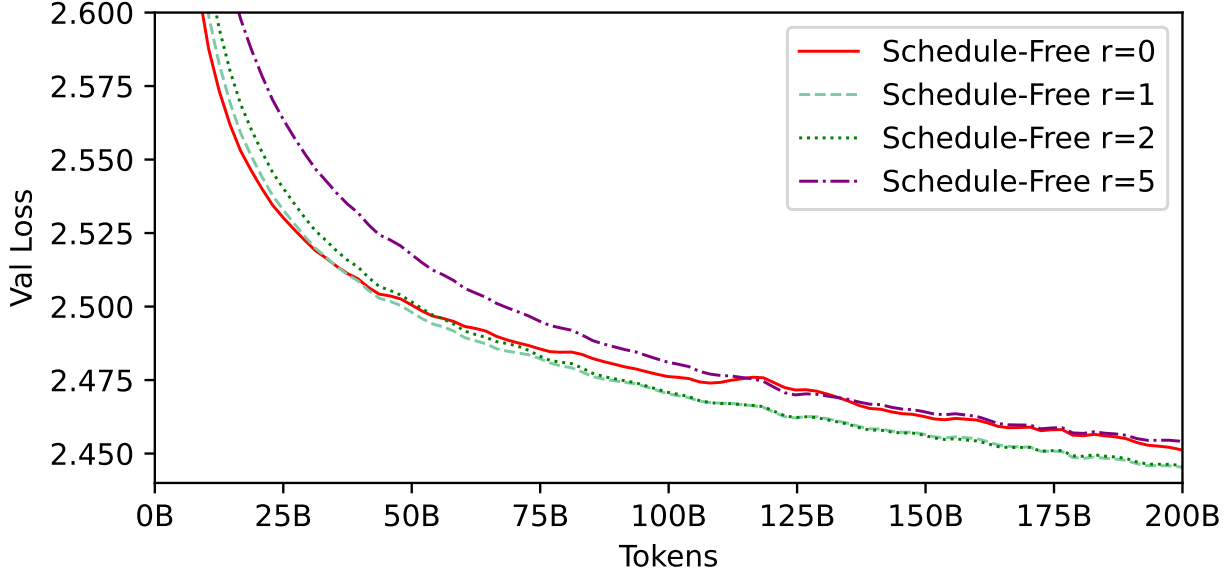


Figure 11 Using values around $r=1$ is beneficial for longer runs, but slightly degrades performance below 30B tokens.

The default is $r = 0$, giving a flat weighting after learning rate warmup. The use of the max of the squared learning rate seen so far is a heuristic choice. The weighting of the time step by power r is theoretically justified, as a convergence rate can be proven for any $r \geq 0$, and for certain problem classes, improved rates are obtained for $r = 1$ when using iterate averaging approaches (Defazio and Gower, 2021).

Figure 11 shows a sweep of r values for a 200B token training run, also using the β annealing technique previously demonstrated in Figure 10b. All other hyper-parameters were kept constant. We see that $r = 0$ (Equal weighting) is slightly better for runs shorter than 30B tokens, but after that $r = 1$ has a very clear and significant advantage. Larger r values than $r = 1$ do not seem to be beneficial. Since $r = 1$ values are typically used for non-strongly convex problems to accelerate convergence, this value is a sensible default for longer runs. Weighting by $r = 1$ when averaging equally spaced checkpoints was found by Li et al. (2025) to improve the performance of the resulting model. **We recommend using $r = 1$ weighting with Schedule-Free for long duration training runs.**

9 Schedule-Free training is HIGHLY predictable

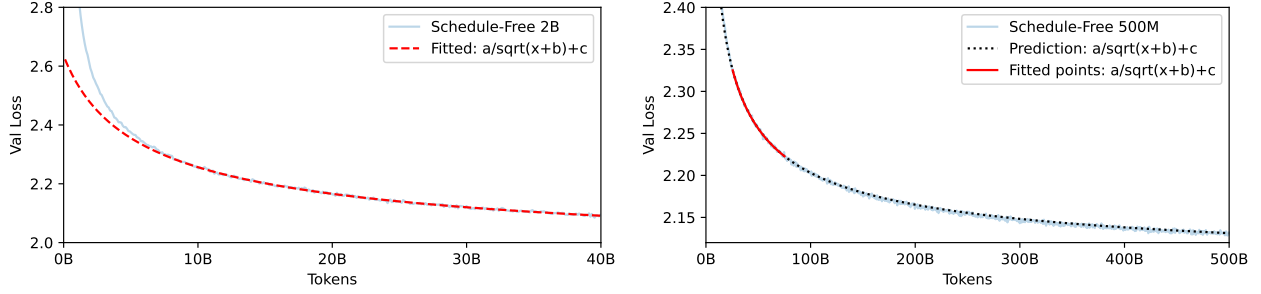
It is well known that stochastic convex optimization problems have a $1/\sqrt{t}$ worst-case convergence rate. In the case of non-smooth G -Lipschitz optimization with SGD iterates z_t , the bound takes the form:

$$\mathbb{E}f(\bar{z}_t) - f_* \leq \frac{\|z_1 - x_*\| G}{\sqrt{t}} \quad (17)$$

When using classical training approaches, this inverse-square-root decay of the loss during training is not seen in practice during real training runs. Normally, this theory-practice gap is hand-waved away, with the theoretical rate considered unrealistic due to one or all of the following factors:

Non-Convexity It seems “unreasonable” that we should expect this convex optimization bound to apply to giant neural network optimization problems. Function value convergence is not even guaranteed for non-convex problems under typical assumptions.

Average Iterate This convergence rate is for the average iterate $\bar{z}_t$, not the actual iterates z_t . In practice for classical training approaches the average iterate is garbage, either it gives NaN, Inf, or terrible loss values. Classical analysis techniques give a $\log(t)$ worse rate for the last iterate (Shamir and Zhang, 2013), or use some sort of tail-averaging.



(a) Schedule-Free Learning loss values fit an inverse-square-root decay rate very well outside an initial burn-in period **(b)** For long training runs, we can use early loss values to predict future loss values with remarkably high accuracy

Figure 12 Schedule-Free loss curves follow a predictable $1/\sqrt{t}$ decay

Worst-case behavior This worst case bound is considered too pessimistic to hold in practice.

We believe that all of these points are misguided, and in fact when using Schedule-Free Learning, we should expect that our convergence behavior should follow Equation 17 extremely closely, at least outside of an initial burn-in phase at the beginning of training. To verify this hypothesis, we performed a curve fitting procedure using `scipy.optimize.curve_fit` with default parameters, fitting the following equation to the sequence of loss values within a single run:

$$f(x_t) = \frac{a}{\sqrt{t+b}} + c \quad (18)$$

Here c approximates f_* , the optimal function value. The offset in the denominator accounts for a changing rate of convergence at the beginning due to learning rate warmup. In Figure 12a, we apply this fit to the last 75% of a short-ish duration (40B tokens) training run of a 2B model. Outside of the initial burn-in period, the fit is *extremely* good.

We can also use this kind of fitting to predict future loss values from early loss values during training. This is particularly useful for very long training runs. In Figure 12b, we train a 500M parameter model for 500B tokens. We use the loss values from 5% to 15% run (dropping the initial 5% burn-in period), to test if we can predict the entire loss curve from just 10% of the training run. As we can see, this fit is also very good, **predicting near identical loss values to the actual measured loss values for the remainder of training**. The curve fit predicts that f_* is 2.072 for this 500M parameter model, compared to 1.887 predicted for the 2B model.

Scaling Ladders

We call the combination of the above approaches the ScheduleFree+ method. We ran a series of scaling ladder experiments, where at each scale we compare against a state-of-the-art Linear Decay schedule, using a learning rate tuned with a grid search. The Schedule-Free version has no learning rate grid search and relies purely on the Polyak Step-size at all scales. The reference implementation of ScheduleFree+ used for these experiments is available at https://github.com/facebookresearch/schedule_free/blob/main/schedulefree/adamc_schedulefree_plus_paper.py.

9.1 Long Duration Training Runs

We found the greatest improvement over schedule-based approaches when training on large numbers of tokens, where beta-annealing and using $r=1$ weighted averaging (Section 8) becomes effective. Figure 13 shows the loss curves and adaptive step sizes at three increasing scales, using 1000 tokens per parameter. This tpp scale is significantly longer than the Chinchilla recommended 20 tokens per parameter, but is more representative of modern training regimes. Tokens per (active) parameter for frontier open models have increased at a rate of $>3x$ per year (Somala and Edelman, 2025). ScheduleFree+ significantly improves upon the AdamW baseline both in terms of final loss, as well as intermediate loss values. Even without beta-annealing, ScheduleFree+ outperforms schedule-based approaches consistently for longer duration training runs (Figure 17). For example, for the 120M model, we ran a grid of longer duration runs to see how long it would take to reach a comparable

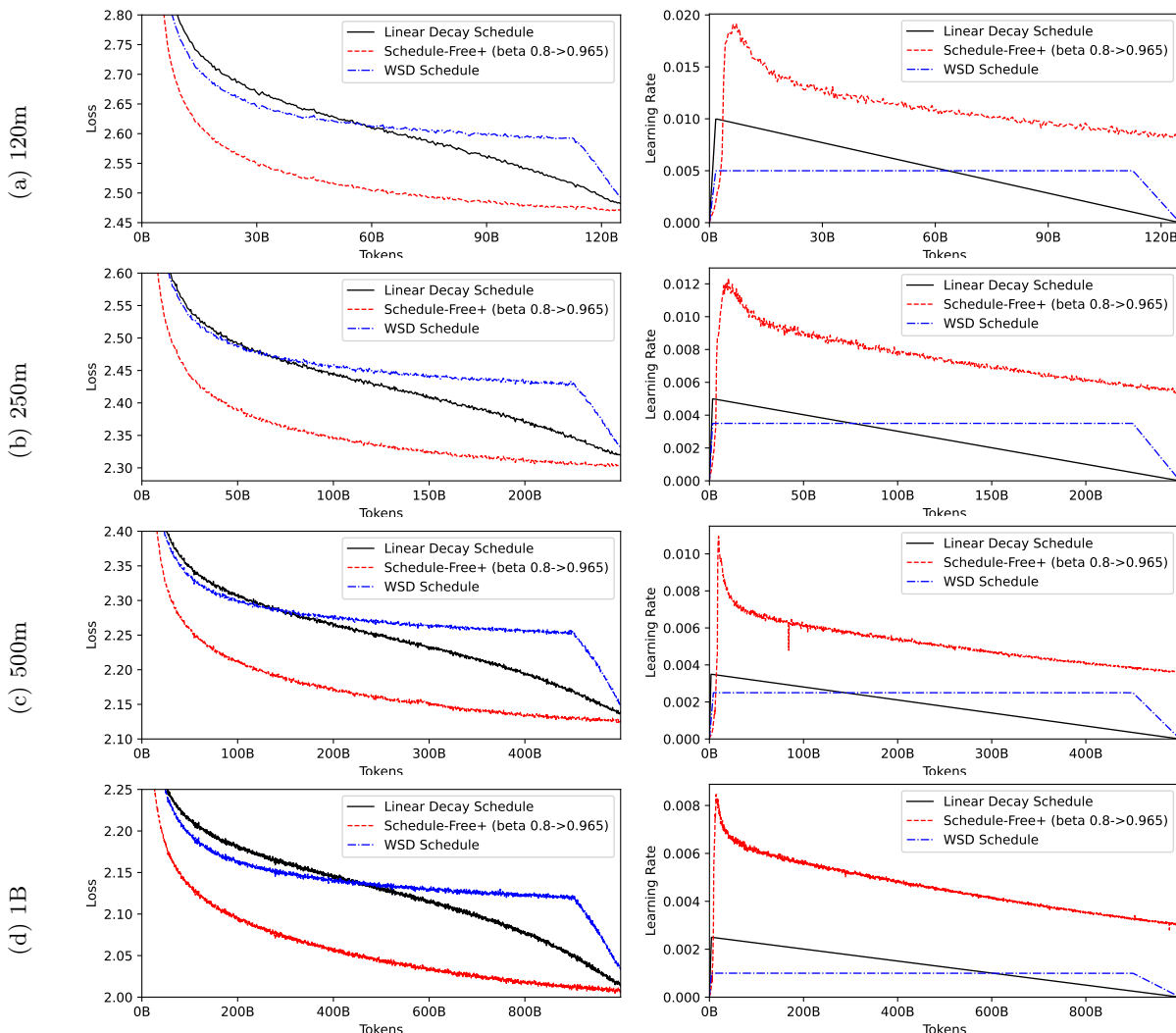


Figure 13 1000 tokens per parameter training runs. Rows (a)-(d) show scaling from 120m to 1B parameters.

loss value using the linear decay schedule. **ScheduleFree+ reached the same final loss value as a 45% longer run**, a 31% reduction in training time to reach the same loss.

Parameters	Tokens	Steps	Batch size	GPUs	Warmup	Beta	r
120M	125B	29,803	4M	8	400	0.8 $\rightarrow$ 0.965	1
250M	250B	59,605	4M	32	400	0.8 $\rightarrow$ 0.965	1
500M	500B	119,210	4M	128	1000	0.8 $\rightarrow$ 0.965	1
1B	1T	238,419	4M	256	1000	0.8 $\rightarrow$ 0.965	1

Table 1 1000 tokens per parameter settings

9.2 Shorter Duration Training Runs

We further tested Schedule-Free at 100 tokens per parameter. At this scale beta-annealing is not effective, and we tested ScheduleFree+ with a fixed $\beta = 0.9$. For models from 120M up to 2B parameters, we find that Schedule-Free exceeds the performance of the WSD schedule at all scales (Figure 14). It also does well against the linear decay schedule, outperforming it at all but the 2B scale.

Parameters	Tokens	Steps	Batch size	GPUs	Warmup	Beta	r
120M	12.5B	2,981	4M	8	400	0.9	0
250M	25B	5,961	4M	8	400	0.9	0
500M	50B	11,921	4M	8	1000	0.9	0
1B	100B	23,842	4M	32	2000	0.9	0
2B	200B	47,684	4M	128	2000	0.9	0

Table 2 100 tokens per parameter settings

Using 20 tokens per parameter still shows an advantage for ScheduleFree+ at smaller model sizes, but we see a small performance gap for larger models when compared to the linear decay schedule. At this token budget, the loss curves for Schedule-Free Learning are not significantly below those from a schedule, which may also contribute to the differences. ScheduleFree+ does however outperform the WSD schedule at all model scales.

At this tpp scale the duration of the run for which gradient norms and weight norms are not at steady state is close to 50% of the run. Schedule-Free takes longer to hit steady state and this contributes to the lower performance for these very short runs. Initializing the weights closer to steady state or using weight projection would help here.

Parameters	Tokens	Steps	Batch size	GPUs	Warmup	Beta	r
120M	2.4B	2,289	1M	8	400	0.9	0
250M	5B	4,769	1M	8	400	0.9	0
500M	10B	9,537	1M	8	1000	0.9	0
1B	20B	19,074	1M	8	2000	0.9	0
2B	40B	38,147	1M	32	2000	0.9	0

Table 3 20 tokens per parameter settings

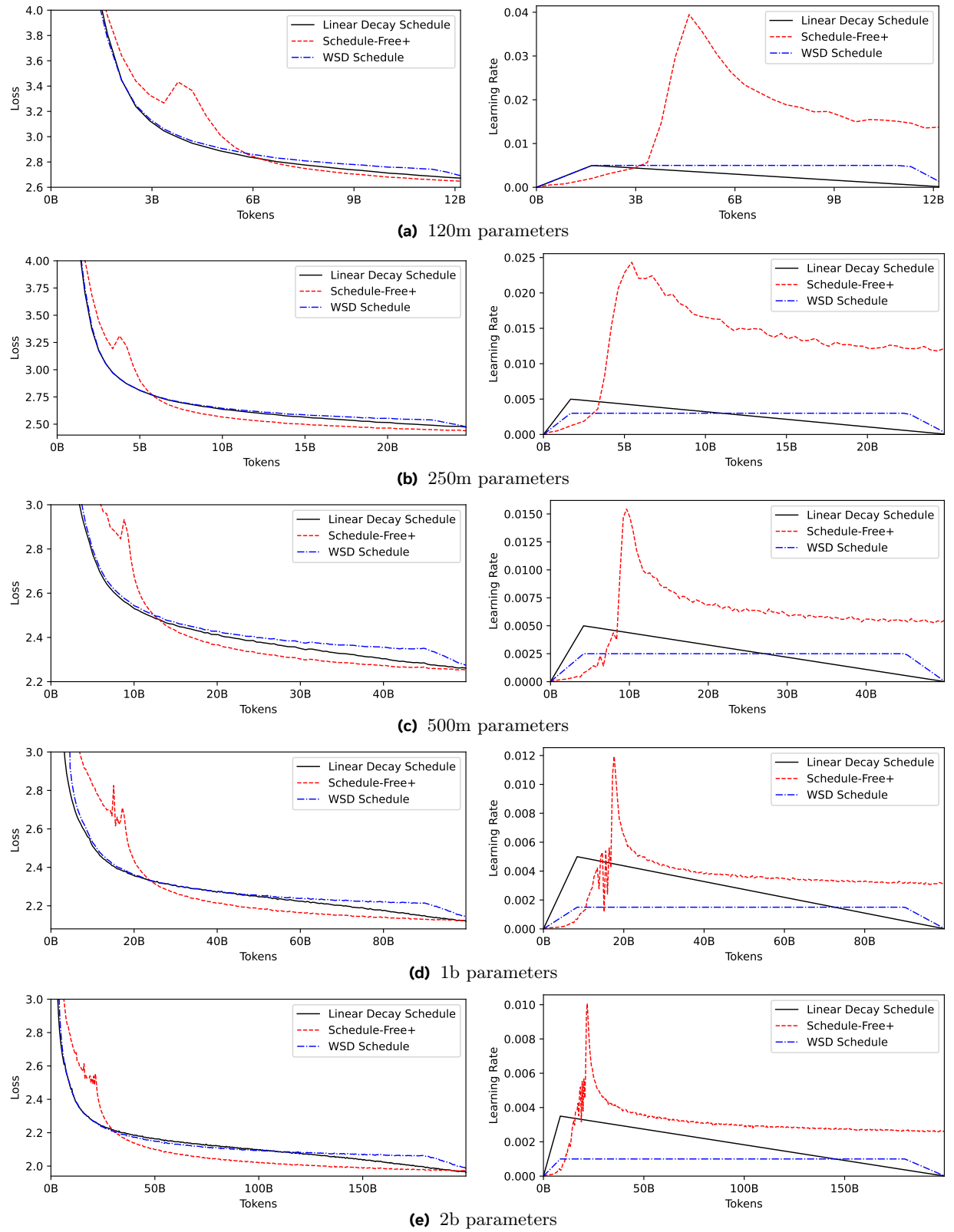


Figure 14 100 tokens per parameter

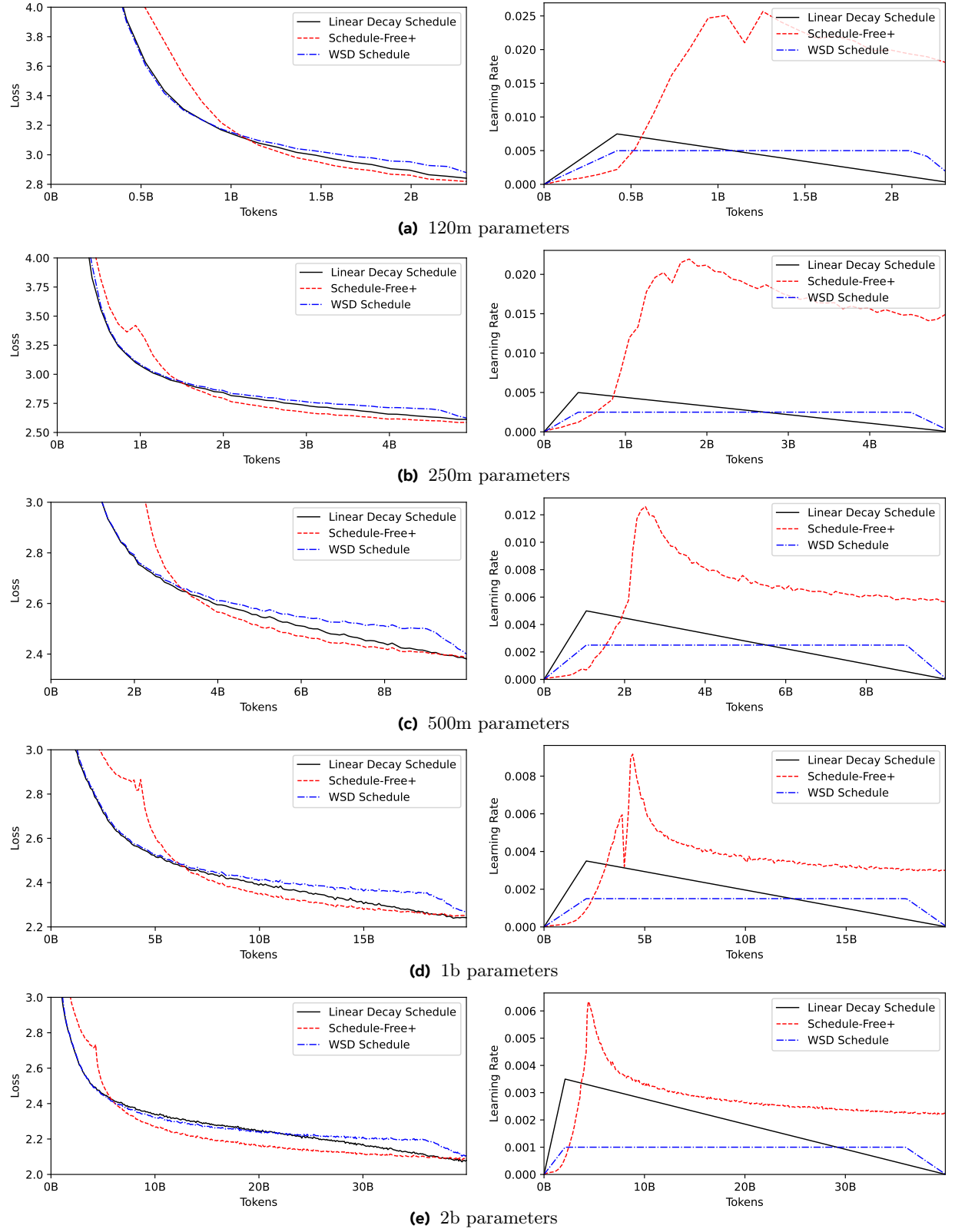


Figure 15 20 tokens per parameter training runs

Algorithm 1 ScheduleFree+ (AdamC + Schedule-Free + Polyak)

Require: Warmup schedule γ_t , weight decay λ , Adam betas (β_1, β_2) , numerical stability constant ϵ

Require: Schedule-Free parameters r, p (**weight_lr_power**), $C_{\text{warmup}}, \beta_{\text{sf}}, \beta_{\text{sf}}^{\text{max}}, T_{\text{anneal}}$

Require: Polyak EMA coefficient β_p

```
1: Initialize parameters  $x_0 = y_0 = z_0 = \theta_0$ 
2: Initialize Adam moments  $m_0 = 0, v_0 = 0$ 
3: Initialize Polyak EMA  $e_0 = 0$ , schedule weight sum  $W_0 = 0$ , max learning rate  $\gamma_{\text{max}} = \epsilon$ 
4: for  $t = 1 \dots T$  do
5:
6:   if  $T_{\text{anneal}} > 0$  then
7:      $\tau_t \leftarrow \min(t/T_{\text{anneal}}, 1)$ 
8:      $\tilde{\beta}_t \leftarrow 1 - \exp((1 - \tau_t) \ln(1 - \beta_{\text{sf}}) + \tau_t \ln(1 - \beta_{\text{sf}}^{\text{max}}))$ 
9:   else
10:     $\tilde{\beta}_t \leftarrow \beta_{\text{sf}}$ 
11:   end if
12:
13:   Evaluate gradient at  $y_{t-1}$ :  $g_t \leftarrow \nabla f(y_{t-1})$ 
14:   Get global function value:  $F_t \leftarrow f(y_{t-1})$ 
15:   Compute global gradient  $L_1$  norm:  $L_1 \leftarrow \|g_t\|_1$ 
16:   Compute inner product correction:  $I_t \leftarrow \beta_t \langle g_t, z_{t-1} - x_{t-1} \rangle$ 
17:   Update Polyak gradient  $L_1$  EMA:  $e_t \leftarrow \beta_p e_{t-1} + (1 - \beta_p) L_1 \sqrt{\pi/2}$ 
18:   Bias-correct  $L_1$  EMA:  $\hat{e}_t \leftarrow e_t / (1 - \beta_p^t)$ 
19:   Compute Polyak scalar:  $\eta_t \leftarrow \max(0, F_t + I_t) / \hat{e}_t$ 
20:   Compute effective learning rate:  $\alpha_t \leftarrow \gamma_t \cdot \eta_t$ 
21:    $\gamma_{\text{max}} \leftarrow \max(\alpha_t, \gamma_{\text{max}})$ 
22:
23:   if  $t \leq C_{\text{warmup}}$  then
24:      $c_t \leftarrow 1$ 
25:   else
26:      $w_t \leftarrow t^r \cdot \gamma_{\text{max}}^p$ 
27:      $W_t \leftarrow W_{t-1} + w_t$ 
28:      $c_t \leftarrow w_t / W_t$ 
29:   end if
30:
31:    $z_t \leftarrow z_{t-1} - \alpha_t^2 \lambda y_{t-1}$ 
32:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
33:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
34:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
35:    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
36:    $z_t \leftarrow z_t - \alpha_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ 
37:    $x_t \leftarrow (1 - c_t) x_{t-1} + c_t z_t$ 
38:    $y_t \leftarrow \tilde{\beta}_t x_t + (1 - \tilde{\beta}_t) z_t$ 
39: end for
40: return  $x_T$ 
```

▷ 1. Schedule-Free Momentum Annealing

▷ 2. Global Metrics & Polyak Step Size

▷ 3. Schedule-Free Averaging Weights

▷ 4. Per-Parameter Updates

▷ Decoupled AdamC weight decay

▷ Update first moment

▷ Update second moment

▷ Bias correction

▷ Adam step on z

▷ Update evaluation point x

▷ Compute next query point y

Discussion

10 Warmup Stable Decay Schedules

The flexibility to halt training early, or to continue to train indefinitely, motivated the introduction of the WSD schedule (Zhai et al., 2022; Hu et al., 2024). WSD schedules start with a standard learning-rate warmup (as used by both Schedule-Free and other schedules), followed by a constant learning rate thereafter. Training can be early stopped by introducing a linear learning rate anneal to 0, typically 10% of the training run is allocated to this annealing phase. WSD-like schedules have been used in some frontier model training runs (Bi et al., 2024; Kimi Team, 2025).

Schedule-Free Learning gives true anytime training; it doesn’t require any additional annealing, a significant advantage over WSD schedules. There is no delay while you wait for the annealing phase before having a usable model to run evaluations on. Restarting training is also much more straight-forward without annealing.

The efficacy of WSD schedules has recently been motivated by a river-valley view of the loss landscape (Wen et al., 2025). In this view, even when the loss curve is not decreasing progress is still being made along ‘rivers’ in the loss landscape, and this progress is revealed during the annealing phase as the high loss due to bouncing off of valley-walls is reduced. Schedule-Free Learning has also recently been considered from this viewpoint (Song et al., 2025), where it’s argued that it navigates rivers without the high-loss bouncing that occurs with the WSD schedule.

It is tempting to believe that the river-valley landscape implies a twisting, winding river and rugged ever-changing mountain ranges, a complex landscape out of reach theoretical analysis. Remarkably, it appears that convex optimization theory can precisely explain the shape of loss curves that arise from the WSD schedule. Schaipp et al. (2025) show that the precise shape of the loss curve can be predicted by the use of schedule-dependent learning rate bounds. Given a schedule η_t with peak learning rate γ , Defazio et al. (2023) show that the loss at every time-step is bounded by the following (rather complicated) expression:

$$E[f(x_t) - f_*] \leq \frac{1}{2\gamma\bar{\eta}_t} \left[D^2 + \gamma^2 \sum_{i=1}^t \eta_i^2 E \|g_i\|^2 \right] + \frac{\gamma}{2} \sum_{k=1}^{t-1} \frac{\eta_k}{\sum_{i=k+1}^t \eta_i} \left(\frac{1}{\sum_{i=k}^t \eta_i} \sum_{i=k}^t \eta_i^2 E \|g_i\|^2 \right), \quad (19)$$

where $\bar{\eta}_t = \sum_{i=1}^t \eta_i$ and D is an estimate of the initial distance to the solution. The right hand side depends on the expected gradient norms at each time-step, however assuming constant gradient norms gives good predictions outside of the initial warmup-phase of optimization. When evaluated numerically, this bound predicts the same loss drops during the annealing phase of the WSD that are seen in practice.

We use the Linear Decay schedule as the baseline horizon-dependent experiments in this work. The Linear Decay schedule arises from optimizing the right-hand-side of Equation 19 to find the optimal schedule when expected gradient norms are flat during training, and Defazio et al. (2023) show that it is a worst case-optimal schedule for non-smooth convex optimization problems. Besides this theoretical motivation, Linear Decay schedules have been observed to give state-of-the-art results across a wide variety of problems, including LLMs (Bergsma et al., 2025) with just a few exceptions such as long duration training runs on ResNet models where classical cosine schedules works well. In the convex case, the river-valley view suggests a small, long (but convex) river surrounded by mountains of varying steepness. The loss landscape along the WSD basin has been empirically observed to exhibit (quasi-)convexity (Belloni et al., 2025).

The WSD schedule’s practical performance is good, but it is consistently outperformed by the linear decay schedule in our experiments when tuned to the same training horizon. The suboptimality of the WSD schedule against properly tuned baseline schedules is not emphasized in the literature, but has been observed by others also (Zhai et al., 2022; Bergsma et al., 2025).

11 Cosine Schedules

In the early era of LLM training, a cosine schedule that decreases the learning rate from its initial optimal value to a 10x smaller value was heavily used, due to its appearance in a series of influential DeepMind papers

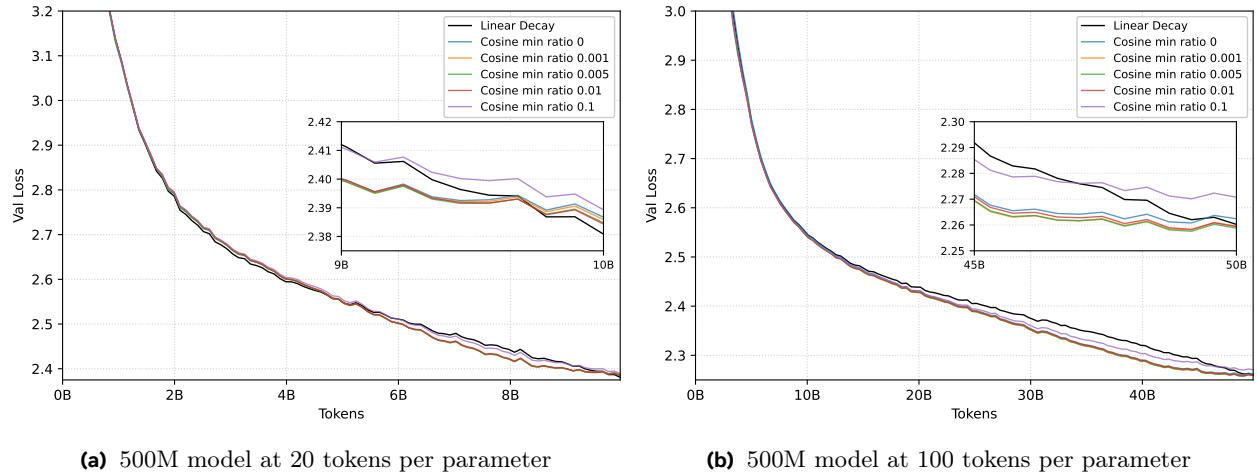


Figure 16 Cosine schedules tie or even under-perform Linear Decay schedules for LLM training

(Rae et al., 2021; Hoffmann et al., 2022). Figure 16 shows a comparison of Linear Decay Schedules and Cosine Schedules when both are tuned by learning rate grid search. For the cosine schedule, we see that a 0.1 ratio between max and min is suboptimal, and there are only tiny differences between the other ratios tested, only visible in the magnified loss plots.

12 Model Merging, Souping and Averaging

Instead of decreasing the learning rate during the later stages of training, multiple checkpoints can be averaged to obtain a lower loss point. This approach was first proposed (to our knowledge) by Sandler et al. (2023), and was more recently explored at small scale by Hägele et al. (2024) and at larger scales by Li et al. (2025). Averaging of model checkpoints has recently been popularized under the name *Model Merging*, but the general idea is closely related to stochastic weight averaging (Izmailov et al., 2018) and classical Polyak averaging (Polyak, 1990; Ruppert, 1988) explored in the convex optimization literature. This link to optimization theory is increasingly de-emphasized in more recent literature, or even lost, yet it provides a crucial theoretical underpinning for these methods.

Weight averaging of neural networks is problematic given the highly nonconvex nature of the loss landscape, and obviously shouldn’t work. Having said that, it has been repeatedly demonstrated to work well in practice (Yang et al., 2024). Averages can even compete with ensembles of multiple models, as demonstrated in the Model Souping approach (Wortsman et al., 2022a). Averaging of base models with their fine-tuned counterparts during post-training has been shown to improve generalization performance (Wortsman et al., 2022b; Ilharco et al., 2022). Averaging can be used to increase parallelism during the training process, with models being trained independently for 100s of steps before merging, as used by DiLoCo (Douillard et al., 2023), or for longer durations on domain-specific data, as done in the Branch-Train-Merge approach (Li et al., 2022).

Schedule-Free differs from the above approaches by *mixing-in* the average point during training; Polyak-like averaging procedures use the average point in a purely off-line fashion, a crucial difference. Without this mixing operation, Schedule-Free Learning performs extremely poorly; it is an absolutely critical component of the method. Checkpoint averaging approaches typically focus on averaging points from the later parts of the training run (sometimes known as tail or suffix averaging), which is more stable. In contrast, Schedule-Free Learning’s interpolation appears to enable averaging of earlier iterates, and generally yields faster convergence.

Conclusion

We demonstrate that Schedule-Free Learning can be adapted to perform well on Large Language Model training tasks, providing a stable, anytime training procedure that outperforms both other anytime procedures such as WSD schedule-training, and also state-of-the-art horizon aware training approaches. The modifications here address limitations of Schedule-Free Learning identified by [Hägele et al. \(2024\)](#) and [Morwani et al. \(2025\)](#). Furthermore, in combination with Adaptive step sizes given by a novel modification of the Polyak step size, we demonstrate fully learning-rate-free learning scales to LLMs.

References

- Belloni, A., Noci, L., and Orvieto, A. (2025). Universal dynamics of warmup stable decay: understanding wsd beyond transformers. In *High-dimensional Learning Dynamics 2025*.
- Bergsma, S., Dey, N. S., Gosal, G., Gray, G., Soboleva, D., and Hestness, J. (2025). Straight to zero: Why linearly decaying the learning rate to zero works best for LLMs. In *The Thirteenth International Conference on Learning Representations*.
- Bi, X., Chen, D., Chen, G., Chen, S., Dai, D., Deng, C., Ding, H., Dong, K., Du, Q., Fu, Z., et al. (2024). Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*.
- Defazio, A. (2025). Why gradients rapidly increase near the end of training.
- Defazio, A., Cutkosky, A., Mehta, H., and Mishchenko, K. (2023). Optimal linear decay learning rate schedules and further refinements.
- Defazio, A. and Gower, R. M. (2021). The power of factorial powers: New parameter settings for (stochastic) optimization. In Balasubramanian, V. N. and Tsang, I., editors, *Proceedings of The 13th Asian Conference on Machine Learning*, volume 157 of *Proceedings of Machine Learning Research*, pages 49–64. PMLR.
- Defazio, A. and Mishchenko, K. (2023). Learning-rate-free learning by D-adaptation. *The 40th International Conference on Machine Learning (ICML 2023)*.
- Defazio, A., Xingyu, Yang, Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. (2024). The road less scheduled.
- Douillard, A., Feng, Q., Rusu, A. A., Chhaparia, R., Donchev, Y., Kuncoro, A., Ranzato, M., Szlam, A., and Shen, J. (2023). Diloco: Distributed low-communication training of language models. *arXiv preprint arXiv:2311.08105*.
- Hägele, A., Bakouch, E., Kosson, A., Allal, L. B., Von Werra, L., and Jaggi, M. (2024). Scaling laws and compute-optimal training beyond fixed training durations. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C., editors, *Advances in Neural Information Processing Systems*, volume 37, pages 76232–76264. Curran Associates, Inc.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. (2022). An empirical analysis of compute-optimal large language model training. *Advances in neural information processing systems*, 35:30016–30030.
- Hu, S., Tu, Y., Han, X., Cui, G., He, C., Zhao, W., Long, X., Zheng, Z., Fang, Y., Huang, Y., Zhang, X., Thai, Z. L., Wang, C., Yao, Y., Zhao, C., Zhou, J., Cai, J., Zhai, Z., Ding, N., Jia, C., Zeng, G., dahai li, Liu, Z., and Sun, M. (2024). MiniCPM: Unveiling the potential of small language models with scalable training strategies. In *First Conference on Language Modeling*.
- Ilharco, G., Wortsman, M., Gadre, S. Y., Song, S., Hajishirzi, H., Kornblith, S., Farhadi, A., and Schmidt, L. (2022). Patching open-vocabulary models by interpolating weights. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, *Advances in Neural Information Processing Systems*, volume 35, pages 29262–29277. Curran Associates, Inc.
- Ivgi, M., Hinder, O., and Carmon, Y. (2023). Dog is sgd’s best friend: A parameter-free dynamic step size schedule. In *International Conference on Machine Learning*, pages 14465–14499. PMLR.
- Izmailov, P., Podoprikin, D., Garipov, T., Vetrov, D. P., and Wilson, A. G. (2018). Averaging weights leads to wider optima and better generalization. In *Conference on Uncertainty in Artificial Intelligence*.
- Khaled, A., Mishchenko, K., and Jin, C. (2023). DoWG unleashed: An efficient universal parameter-free gradient descent method. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Kimi Team (2025). Kimi k2: Open agentic intelligence.
- Kosson, A., Messmer, B., and Jaggi, M. (2024). Rotational equilibrium: How weight decay balances learning across neural networks.
- Kosson, A., Welborn, J., Liu, Y., Jaggi, M., and Chen, X. (2025). Weight decay may matter more than mup for learning rate transfer in practice.
- Li, M., Gururangan, S., Dettmers, T., Lewis, M., Althoff, T., Smith, N. A., and Zettlemoyer, L. (2022). Branch-train-merge: Embarrassingly parallel training of expert language models. *arXiv preprint arXiv:2208.03306*.

- Li, Y., Ma, Y., Yan, S., Zhang, C., Liu, J., Lu, J., Xu, Z., Chen, M., Wang, M., Zhan, S., et al. (2025). Model merging in pre-training of large language models. *arXiv preprint arXiv:2505.12082*.
- Marek, M., Lotfi, S., Somasundaram, A., Wilson, A. G., and Goldblum, M. (2025). Small batch size training for language models: When vanilla sgd works, and why gradient accumulation is wasteful. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc.
- Mishchenko, K. and Defazio, A. (2023). Prodigy: An expeditiously adaptive parameter-free learner.
- Morwani, D., Vyas, N., Zhang, H., and Kakade, S. (2025). Connections between schedule-free optimizers, ademamix, and accelerated sgd variants. *arXiv preprint arXiv:2502.02431*.
- Oikonomou, D., Buchholz, M., Pun, Y.-M., Gower, R. M., and Loizou, N. (2026). Taking the road less scheduled with adaptive polyak steps.
- Polyak, B. T. (1990). A new method of stochastic approximation type. *Avtomatika i telemekhanika*, (7):98–107.
- Rae, J. W., Borgeaud, S., Cai, T., Millican, K., Hoffmann, J., Song, F., Aslanides, J., Henderson, S., Ring, R., Young, S., et al. (2021). Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*.
- Ruppert, D. (1988). Efficient estimations from a slowly convergent Robbins-Monro process. *Technical Report, Cornell University*.
- Sandler, M., Zhmoginov, A., Vladymyrov, M., and Miller, N. (2023). Training trajectories, mini-batch losses and the curious role of the learning rate. *arXiv preprint arXiv:2301.02312*.
- Schaipp, F., Hägele, A., Taylor, A., Simsekli, U., and Bach, F. (2025). The surprising agreement between convex optimization theory and learning-rate scheduling for large model training. In *Forty-second International Conference on Machine Learning*.
- Shallue, C. J., Lee, J., Antognini, J., Sohl-Dickstein, J., Frostig, R., and Dahl, G. E. (2019). Measuring the effects of data parallelism on neural network training. *Journal of Machine Learning Research*, 20(112):1–49.
- Shamir, O. and Zhang, T. (2013). Stochastic gradient descent for non-smooth optimization: Convergence results and optimal averaging schemes. In *International conference on machine learning*, pages 71–79. PMLR.
- Somala, V. and Edelman, Y. (2025). Training open-weight models is becoming more data intensive. Accessed: 2026-01-14.
- Song, M., Baek, B., Ahn, K., and Yun, C. (2025). Through the river: Understanding the benefit of schedule-free methods for language model training. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Van Laarhoven, T. (2017). L2 regularization versus batch and weight normalization. *arXiv preprint arXiv:1706.05350*.
- Wen, K., Li, Z., Wang, J. S., Hall, D. L. W., Liang, P., and Ma, T. (2025). Understanding warmup-stable-decay learning rates: A river valley loss landscape view. In *The Thirteenth International Conference on Learning Representations*.
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., and Schmidt, L. (2022a). Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S., editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 23965–23998. PMLR.
- Wortsman, M., Ilharco, G., Kim, J. W., Li, M., Kornblith, S., Roelofs, R., Lopes, R. G., Hajishirzi, H., Farhadi, A., Namkoong, H., and Schmidt, L. (2022b). Robust fine-tuning of zero-shot models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7959–7971.
- Yang, E., Shen, L., Guo, G., Wang, X., Cao, X., Zhang, J., and Tao, D. (2024). Model merging in llms, mllms, and beyond: Methods, theories, applications, and opportunities. *ACM Computing Surveys*.
- Zhai, X., Kolesnikov, A., Houlsby, N., and Beyer, L. (2022). Scaling vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12104–12113.
- Zhang, G., Li, L., Nado, Z., Martens, J., Sachdeva, S., Dahl, G., Shallue, C., and Grosse, R. B. (2019). Which algorithmic choices matter at which batch sizes? insights from a noisy quadratic model. *Advances in neural information processing systems*, 32.

- Zhang, H., Morwani, D., Vyas, N., Wu, J., Zou, D., Ghai, U., Foster, D., and Kakade, S. (2024). How does critical batch size scale in pre-training? *arXiv preprint arXiv:2410.21676*.
- Zinkevich, M. (2003). Online convex programming and generalized infinitesimal gradient ascent. In *Proceedings of the Twentieth International Conference on International Conference on Machine Learning*, pages 928–935.

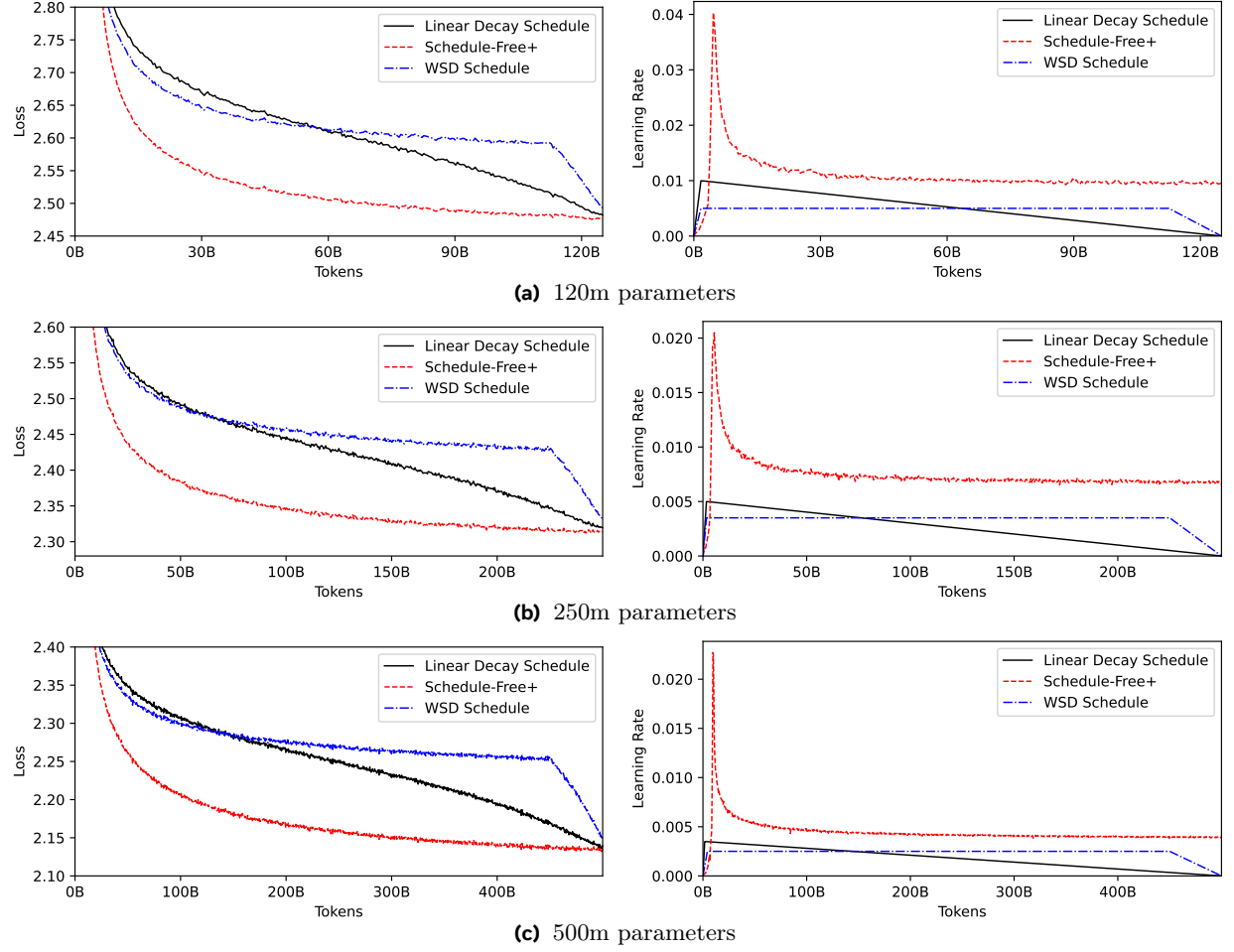


Figure 17 1000 tokens per parameter without beta-annealing

A Additional Plots

B Model Architecture Settings

	120M	250M	500M	1B	2B
dim	768	1024	1280	2048	2560
layers	12	14	21	16	23
heads	12	16	20	32	32
ffn exp	4	4	4	4	4
ff multiplier	256	256	256	256	256
rope theta	10000	10000	10000	10000	10000

C Configuration Used for Each Figure

C.1 Figure 1

ScheduleFree+ runs here include the Polyak step size, warm starting, $r = 1$ and beta annealing. A 500M parameter model was used.

Method	Linear Decay	WSD	ScheduleFree+
Tokens	500B	500B	500B
Sequence Length	1024	1024	1024
Batch-size	4M	4M	4M
GPUs	128	128	128
Steps	119,210	119,210	119,210
Warmup	1000	1000	1000
Learning Rate	0.0035	0.0025	N/A
Decay	0.05	0.05	20
Clipping	1.0	1.0	1.0
r	N/A	N/A	1.0

C.2 Figure 2 a

Batch-size was varied directly up to 128 samples per gpu, above that gradient accumulation was used to gather larger effective batch sizes. A 120M parameter model was used.

Method	AdamW	Schedule-Free
Tokens	7.8B	7.8B
Batch-size	Varies	Varies
Sequence Length	1024	1024
GPUs	8	8
Steps	15,000	15,000
Warmup	50,000/bs	50,000/bs
Warm-started	Yes	Yes
Learning Rate	$0.01 \cdot \sqrt{\text{bs}/64}$	$0.02 \cdot \sqrt{\text{bs}/32}$
Decay	0.05	0.002
Clipping	10	10
Schedule	Linear Decay	N/A
SF Beta	N/A	0.9

C.3 Figure 2 b

Method	AdamW	Schedule-Free
Tokens	7.8B	7.8B
Batch-size	Varies	Varies
Sequence Length	1024	1024
GPUs	8	8
Steps	15,000	15,000
Warmup	50,000/bs	50,000/bs
Warm-started	Yes	Yes
Learning Rate	$0.01 \cdot \sqrt{\text{bs}/64}$	$0.015 \cdot \sqrt{\text{bs}/32}$
Decay	0.05	0.002
Clipping	10	10
Schedule	Linear Decay	N/A
SF Beta	N/A	0.9
AdamW Beta1	0.9	0.75

C.4 Figure 3 a

Batch size of 0.5M was used, with weight decay 0.005. Otherwise hyper-parameters match the previous figure.

C.5 Figure 3 b

Method	SF Refinement	SF Outer Mom
Tokens	7.8B	7.8B
Batch-size	Varies	Varies
Sequence Length	1024	1024
GPUs	8	8
Steps	15,000	15,000
Warmup	50,000/bs	50,000/bs
Warm-started	Yes	Yes
Learning Rate	$0.01 \cdot \sqrt{\text{bs}/32}$	$0.015 \cdot \sqrt{\text{bs}/32}$
Decay	0.002	0.002
Clipping	10	10
SF Beta	N/A	0.9
AdamW Beta1	0.0	0.9
Refinement C	50	N/A

C.6 Figure 4 a & b

A 120M parameter model was used.

Method	AdamW	AdamC	SF WD 0.002	SF WD 0.05
Tokens	7.8B	7.8B	7.8B	7.8B
Batch-size	1M	1M	1M	1M
Sequence Length	1024	1024	1024	1024
GPUs	8	8	8	8
Steps	15,000	15,000	15,000	15,000
Warmup	1000	1000	1000	1000
Warm-started	Yes	Yes	Yes	Yes
Learning Rate	0.015	0.015	0.03	0.03
Decay	0.05	0.05	0.002	0.05
Clipping	10	10	10	10
Schedule	Linear Decay	Linear Decay	N/A	N/A
AdamW Beta	0.9	0.9	0.75	0.75
SF Beta	N/A	N/A	0.9	0.9

C.7 Figure 5 a, c & d

Weights norms are shown from Figure 4's runs.

C.8 Figure 6

Configuration matches previous figure. 1/L1 runs use learning rate multiplier of 2.

C.9 Figure 7

Method	ScheduleFree+
Parameters	2B
Tokens	40B
Batch-size	1M
Sequence Length	1024
GPUs	32
Steps	38147
Warmup	2000
C-Warmup	2000
Warm-started	No
Learning Rate	Polyak
Decay	50
Clipping	1
SF Beta	0.9
AdamW Beta1	0.9

C.10 Figure 8

Method	ScheduleFree Polyak	ScheduleFree 1/L1
Parameters	120M	120M
Tokens	10.5B	10.5B
Batch-size	0.5M	0.5M
Sequence Length	1024	1024
GPUs	8	8
Steps	20,000	20,000
Warmup	781	781
C-Warmup	781	781
Warm-started	No	No
Learning Rate	Polyak	Varies
Decay	5	5
Clipping	1	1
SF Beta	0.9	0.9
AdamW Beta1	0.9	0.9

C.11 Figure 9

Method	ScheduleFree+
Parameters	250M
Tokens	5B
Batch-size	1M
Sequence Length	1024
GPUs	8
Steps	4769
Warmup	400
C-Warmup	Varies
Warm-started	No
Learning Rate	Polyak
Decay	5
Clipping	1
SF Beta	0.9
AdamW Beta1	0.9

C.12 Figure 10

Method	ScheduleFree+
Parameters	120M
Tokens	200B
Batch-size	0.5M
Sequence Length	1024
GPUs	8
Steps	400,000
Warmup	0
C-Warmup	0
Warm-started	Yes
Learning Rate	Polyak
Decay	5
Beta2	0.98
Clipping	10
SF Beta	Varies
AdamW Beta1	0.9
r	1

C.13 Figure 11

Configuration matches previous figure with SF beta annealed from 0.9 to 0.965.

C.14 Figure 12 a

Fit parameters $a = 41.5$, $b = 3076$, $c = 1.88$

Method	ScheduleFree+
Parameters	2B
Tokens	40B
Batch-size	1M
Sequence Length	1024
GPUs	32
Steps	38147
Warmup	400
C-Warmup	400
Warm-started	No
Learning Rate	Polyak
Decay	50
Clipping	1
SF Beta	0.9
AdamW Beta1	0.9
r	0

C.15 Figure 12 b

Fit parameters $a = 20.3$, $b = 377$, $c = 2.07$

Method	ScheduleFree+
Parameters	500M
Tokens	500B
Batch-size	4M
Sequence Length	1024
GPUs	128
Steps	119,210
Warmup	400
C-Warmup	400
Warm-started	No
Learning Rate	Polyak
Decay	20
Clipping	1
SF Beta	0.9
AdamW Beta1	0.9
r	1